Do Coding Agents Introduce SOLID Violations? An Empirical Study of Agent Pull Requests in Java and Python

Akram Ben Fekih
ISSAT Sousse, University of Sousse
Sousse, Tunisia
akram.bf.dev@gmail.com

Maha Khemaja
ISSAT Sousse, University of Sousse
Sousse, Tunisia
maha.khemaja@issatso.u-sousse.tn

[Third Author]
[Institution]
[City], [Country]
[email]

Abstract

Coding agents such as Claude Code, GitHub Copilot and OpenAI Codex now submit pull requests to open-source projects, but they are mostly evaluated on whether their code passes tests. We examine whether their contributions follow the five SOLID design principles, using 9,191 analysable pull requests from the AIDev dataset (4,245 in Java and 4,946 in Python, merged, rejected and open). For each principle we built a deterministic detector and ran it on the repository before and after each pull request, so that a pull request is charged only with the violations it adds. An LLM judge labelled 2,475 sampled entities to estimate the precision and recall of each detector, and the Java detectors were also compared with CK, PMD and DesigniteJava. Two detectors, new import cycles (Dependency Inversion) and refused bequest (Liskov Substitution), had high precision. The other three had precision between 0.110 and 0.425 and are used only as screens. Precision-adjusted estimates suggest that 0.4–4.1% of Java and 0.3–3.5% of Python pull requests introduce a violation of a given principle, while the touched code already contains many more. The number of files changed was the strongest predictor of a flag, and the only agent difference found in both languages was a higher Open/Closed flag rate for Claude Code. All results are estimates for these detectors and rest on agreement with an LLM judge, which is not ground truth.

CCS Concepts

• **Software and its engineering** → **Maintaining software**; • **General and reference** → *Empirical studies.*

Keywords

coding agents, SOLID principles, software design, static analysis, mining software repositories, LLM-as-a-judge

1 Introduction

AI coding tools no longer only suggest the next line of code. Agents such as Claude Code, the GitHub Copilot coding agent and OpenAI Codex take a task, edit several files, run the tests and open a pull request. The AIDev dataset [13] contains 7,685,281 pull requests opened by such agents in 957,209 public GitHub repositories. Many of these pull requests are merged, and after a merge most of the maintenance on agent-written files is done by people [24]. The design of agent code matters to the projects that accept it.

Most evaluations of coding agents measure functional correctness. HumanEval [3], BigCodeBench [33] and SWE-bench [9] check whether generated code passes a test suite, usually for a single function or issue. Passing tests says little about coupling, cohesion or class hierarchies, and it is a weak predictor of whether maintainers would accept a change: METR found that roughly half of the agent patches that pass SWE-bench Verified would not be merged [28]. Some recent studies look at structure. Pehlivan et al. [22] asked language models to detect SOLID violations in curated examples, Kashif et al. [10] scanned projects generated with Cursor for design issues, SlopCodeBench [21] measured erosion over repeated tasks, and NITR [31] asked agents to fix maintainability problems. These studies use curated examples, benchmark tasks or newly generated projects, or they count tool warnings without checking how many are real problems. None of the studies we reviewed (Table 1) measures each SOLID principle on real contributions by several agents with a detector whose precision and recall are reported.

Measuring design in pull requests raises two problems. A pull request changes code that already exists, so a violation found in the changed files may be older than the pull request. In addition, the SOLID principles [17, 18] describe intent (for example, a class should have one reason to change), while a static detector sees only structure. How well a detector matches the principle has to be measured before its numbers can be interpreted.

In this study we sampled 14,006 pull requests by Claude Code, GitHub Copilot and OpenAI Codex from AIDev, including merged, rejected and open ones. After removing those that changed no analysable source file or whose source trees could not be recovered, 9,191 remained: 4,245 in Java and 4,946 in Python. We wrote one deterministic detector per principle and ran it on the repository before and after each pull request, charging the pull request only with the difference. To estimate how far the detectors can be trusted, an LLM judge read 2,475 sampled entities and labelled each one with a reason, following the approach evaluated by Pehlivan et al. [22]. For Java we also compared the detectors with three existing analysers. We repeated the whole study in Python, where classes are optional and no compiler checks overrides, and we treat a result as general only when it appears in both languages. The study addresses two research questions:

- *RQ1* (Adherence and differentiation): Do coding agents differ in how well their pull requests adhere to the five SOLID principles, and how many of the design problems in the code they touch do they introduce themselves, in Java and in Python?
- *RQ2* (Measurability): Can SOLID adherence be measured at repository scale with deterministic structural detectors, and how well do these detectors agree with an independent reading judge and with established analysers?

RQ1 is answered by three analyses: differences between agents once change size, repository and time are taken into account; a comparison of merged and unmerged code; and a comparison of

the violations a pull request introduced with those already present in the code it touched. Each analysis is carried out in both languages.

Two of the five detectors, new import cycles for Dependency Inversion and refused bequest for Liskov Substitution, had high precision (in Java, 0.892 for import cycles and 64 of 64 for refused bequest). The other three had precision between 0.110 and 0.425. Precision-adjusted estimates suggest that few pull requests introduced a violation, although the touched code already contained many. Change size accounted for most of the differences between agents. The one difference that held in both languages was a higher Open/Closed rate for Claude Code (odds ratio 2.22 in Java and 2.09 in Python). Merged and unreviewed Java code looked the same, while merged Python code had fewer flags than unmerged Python code.

The main contributions of this paper are:

- five deterministic SOLID detectors for Java and Python that compare the repository before and after a pull request;
- precision and stratum-weighted recall for each detector, a holdout test of two refinements, a test of both refinements in a second language, and a comparison with CK, PMD and DesigniteJava;
- a comparison of three agents over 9,191 analysable pull requests in two languages;
- evidence that the Liskov violations that can occur depend on the compiler (46 of the 51 confirmed Python violations are faults that `javac` rejects), and that a categorical detector rule held on a Java holdout while a fitted numeric threshold did not, with the same pattern in small Python samples.

The pipeline, the labels and the analysis scripts are available at [replication package URL].

2 Background and Related Work

2.1 SOLID and its detection

Martin's design principles [17, 18] are usually grouped under the acronym SOLID. The Single Responsibility Principle (SRP) says a class should have one reason to change. The Open/Closed Principle (OCP) says a module should be open for extension and closed for modification. The Liskov Substitution Principle (LSP) requires that a subtype can be used wherever its base type is expected. The Interface Segregation Principle (ISP) says clients should not depend on members they do not use, and the Dependency Inversion Principle (DIP) says modules should depend on abstractions.

Violations are usually detected through code smells [7, 26] and metric-based detection strategies [16], which combine thresholds on metrics such as the CK suite [4], LCOM4 [8] and conceptual cohesion [15]. Classes involved in antipatterns tend to change more and to contain more faults [11], but Olbrich et al. [20] found that God Classes are less change-prone and fault-prone than other classes once size is controlled. Flags from size-based detectors need to be checked against the code.

2.2 Design quality of AI-generated code

Pehlivan et al. [22] prompted four language models to detect SOLID violations in 240 validated examples and found SRP the easiest principle to detect and DIP the hardest. Kashif et al. [10] generated projects with Cursor averaging 16,965 lines. The projects reached 91% functional correctness but had 1,305 CodeScene and 3,193 SonarQube design issues. Zhu et al. [32] report that code volume predicts structural degradation in AI-generated repositories, which they call a Volume-Quality Inverse Law. SlopCodeBench [21] followed 15 agents through 36 iterative problems and found structural erosion increasing in 77% of trajectories. In NITR [31], agents fixed 4.3% of dependency-control problems and 15.2% of responsibility-decomposition problems across 21 C++ probes. Konrad et al. [12] describe how agents make architectural decisions implicitly, based on how the prompt is worded. These studies work with benchmark tasks or newly generated projects, mostly in a single language, and none of them reports SOLID adherence per principle for several agents.

2.3 Studies of agent pull requests

Several studies use AIDev [13]. Mazloomzadeh et al. [19] compared agentic and human pull requests over time. Cynthia et al. [6] analysed 1,210 merged agent bug-fix pull requests in Python with SonarQube and found that differences between agents mostly disappeared after normalising for code churn. Ben Sghaier et al. [2] showed that the agent software affects code quality even when the underlying model is the same, which suggests that agents built on similar models can still differ. Table 1 summarises the studies closest to ours and the part of our question each one leaves open. Our study measures adherence to each SOLID principle, includes rejected and open pull requests as well as merged ones, covers Java and Python, and reports how valid each detector is.

3 Study Design

This is an observational study. The unit of analysis is the pull request, and we measure how each pull request changed the design of the code it touched. This section describes the corpus, the detectors and the labelling procedure. The analysis specific to each research question is described with its results in Section 4.

3.1 Corpus

We used the AIDev-7.6M release [13]. The default tables of this release cover only repositories with more than 100 stars, so we used the complete tables (`all_pull_request` and `all_repository`). We kept pull requests that change Java or Python source files in repositories with at least 25 stars and a permissive licence (MIT, Apache-2.0, BSD-2-Clause, BSD-3-Clause, ISC, 0BSD, Unlicense, CC0-1.0 or MPL-2.0). Three agents have enough pull requests in both languages: Claude Code, GitHub Copilot and OpenAI Codex. Cursor and Devin were left out because they have too few Java pull requests (94 and 6), and Google Jules because it appears for the first time in this release, so its attribution cannot be compared with an earlier one.

A pull request is merged, rejected (closed without merging) or still open. In AIDev as a whole, 80.9% of agent pull requests are merged, 9.5% are closed without merging and 9.6% are open. We over-sampled rejected and open pull requests so that the outcomes could be compared. For each agent, we drew twice as many merged pull requests as rejected ones. Rejected and open Java pull requests are scarce, so all eligible ones were included, which gave 6,956

Table 1: Closely related studies and what they leave open for the question of this paper.

Study	Code studied	What is measured	Open for our question
Pehlivan et al. [22]	240 curated examples, 4 LLMs	LLM detection of SOLID violations	detection on examples, not agent contributions
Kashif et al. [10]	projects generated with Cursor	CodeScene and SonarQube design issues	one tool, no comparison between agents
Zhu et al. [32]	LLM- and agent-driven projects	code and architecture smells	smells, not principles
SlopCodeBench [21]	15 agents on iterative benchmark problems	verbosity and erosion across checkpoints	benchmark tasks, not SOLID-specific
NITR [31]	21 C++ probes	maintainability tasks with a structural oracle	benchmark tasks in one language
Cynthia et al. [6]	1,210 merged agent bug-fix pull requests (AIDev, Python)	SonarQube issues introduced by the pull request	merged bug fixes only, no SOLID measure
This study	9,191 merged, rejected and open agent pull requests (AIDev, Java and Python)	five SOLID detectors, with measured precision and recall	

Table 2: Analysable pull requests by agent and outcome.

Language	Agent	Merged	Rejected	Open	Total
Java	Claude Code	603	296	210	1,109
	GitHub Copilot	1,491	653	222	2,366
	OpenAI Codex	479	228	63	770
	all	2,573	1,177	495	4,245
Python	Claude Code	940	540	182	1,662
	GitHub Copilot	885	407	178	1,470
	OpenAI Codex	1,069	549	196	1,814
	all	2,894	1,496	556	4,946

Java: 4,245 of 6,956 sampled pull requests, 790 repositories. Python: 4,946 of 7,050 sampled pull requests, 2,921 repositories.

Java pull requests from 1,010 repositories. In Python we drew 1,400 merged, 700 rejected and 250 open pull requests per agent, stratified by repository so that no repository contributes more than 0.4% of any agent and outcome cell. This gave 7,050 Python pull requests from 3,974 repositories. Rates per outcome can be read as reported, but a rate pooled over outcomes reflects this sampling design and not the AIDev population.

Most rejected pull requests were closed without any review. Only two groups record a judgement by a maintainer: pull requests where changes were requested, and pull requests that were reviewed and then closed. We report these groups separately.

3.2 Before and after trees

For each pull request we need the source tree before and after the change. We tried four methods in order: (1) the merge commit and its first parent, (2) the commits of the pull request, (3) the `refs/pull/N/head` reference and its merge base, and (4) the base and head commits reported by the GitHub API. The first method is only available for merged pull requests. Comparisons between merged and unmerged code use only pull requests recovered with methods 2 to 4.

Table 2 shows the pull requests that could be analysed. Of the Java sample, 2,540 pull requests changed no production Java file, 150 exceeded the parsing limit, 18 had lost their history and 3 had no distinct before tree. Of the Python sample, 1,816 changed no Python file that could be analysed, 232 changed files that could not be resolved to a module, 40 had no distinct before tree, 14 exceeded the parsing limit and 2 could not be archived. This includes 496 Python pull requests (7.0% of the sample) whose branch had been deleted, so their source could no longer be retrieved.

3.3 Fact extraction

Java files were parsed with JavaParser and its symbol solver (version 3.25.10), without building the projects. Source roots were inferred from `package` declarations, and the median share of resolved supertypes per pull request is 87.0%. Python files were parsed with the standard `ast` module, and base classes and imports were resolved by module path inside the repository. Both extractors produce the same record of types, members, supertypes, imports and member accesses, and the detectors read only this record. Apart from the extractors and the thresholds, the two languages use the same detector code, labelling procedure and statistical model.

3.4 Detectors

Table 3 summarises the five detectors. Each detector is a pure function of the extracted facts, so the same tree always produces the same flags. The rules are our operational definitions. They capture structures that violations of each principle often leave in the code, and they do not define the principles themselves. Every rate in this paper is therefore a rate for these detectors.

SRP. A class gets one hit for each of six size, complexity and cohesion thresholds it reaches. Cohesion is LCOM4 [8]: methods are nodes, two methods are connected when they use the same field or one calls the other, and constructors are left out. With c connected components and n methods, LCOM $= (c-1)/(n-1)$. A class is flagged with two or more hits. The exception is a class with exactly two hits whose conceptual cohesion C3 is at least 0.4. C3 is the mean cosine similarity between the TF-IDF vectors of the terms in each pair of methods [15]. SRP is scored on the after tree, separately for classes the pull request created and existing classes it edited.

OCP. A dispatch chain is an `if`/`else if` ladder or a `switch` that branches on one subject. Chains that test types (`instanceof`, class literals, `getClass()`) are flagged from three branches. Chains that test a tag or value are flagged from eight branches in Java and five in Python. Chains over an enumeration are ignored, since client code cannot add values to an enumeration. A pull request is flagged when it adds such a chain or adds branches to one.

LSP. Each overriding method is compared with the nearest ancestor method that has a real body, because an abstract method has no behaviour that could be refused. The Java compiler already rules out several faults: a changed parameter list makes an overload, parameter types cannot be narrowed, and visibility cannot

Table 3: The five detectors and their thresholds in each language.

Principle	Unit	Rule	Java	Python
SRP	class (after tree)	One hit per threshold reached by class length, weighted methods, method count, largest method complexity, largest parameter count and LCOM; flag at ≥2 hits, unless exactly 2 hits and C3 ≥ 0.4	362 / 45 / 20 / 11 / 5 / 0.86	250 / 36 / 10 / 12 / 7 / 0.8
OCP	dispatch chain	A type or tag/value chain (`if/else if` ladder or `switch` on one subject) reaching the branch threshold, not over an enumeration; the pull request creates or widens it	type ≥3, tag ≥8	type ≥3, value ≥5
LSP	override pair	Override compared with the nearest ancestor method that has a real body	refused bequest, strengthened precondition, weakened postcondition	refused bequest, widened return; removed, added required or narrowed parameter
ISP	contract	Interface or abstract contract with ≥1 abstract member and ≥1 implementer in the repository; forced stub: ≥2 stubs forming ≥ half of an implementer's members	new or widened contract of ≥13 members, or a forced stub	two of: >7 abstract members, implementer coverage <0.75, a forced stub
DIP	pull request	Two modules become mutually reachable in the internal import graph (Tarjan's strongly connected components), one of them touched	new cyclic pair through a touched file	new cyclic pair through a touched module

SRP thresholds are class lines / weighted methods per class / methods / method complexity / parameters / LCOM. Each threshold is close to the 90th percentile of its own corpus and was set before any label existed.

be reduced. The Java detector therefore checks only refused bequest (the override only throws a refusal exception), a strengthened precondition and a weakened postcondition. In Python, where no compiler checks overrides, the detector also checks removed parameters, added required parameters, narrowed parameter types and widened return types.

ISP. In Java, contracts are interfaces and abstract classes. In Python they are subclasses of `abc.ABC` with abstract members and `Protocol` classes. The width of a contract is its number of abstract members, and each implementer has a stub ratio. The Python flag requires two signals, so Java and Python ISP rates measure different things and are not compared directly.

DIP. We build the internal import graph before and after the change and find strongly connected components with Tarjan's algorithm [27]. A pull request is flagged when two modules become mutually reachable and one of them is a file or module the pull request changed.

All thresholds were set close to the 90th percentile of each language's corpus before any labelling started. After calibration we deleted the results and ran the analysis again, so every flag was produced with the final thresholds. Percentile thresholds are common in metric-based detection strategies [16], but a value in the top decile only marks unusual code, which is not a violation in itself. We therefore treat the thresholds as screening choices and assess them through the precision and recall measured for RQ2.

3.5 Before/after comparison

For OCP, LSP, ISP and DIP, the detector runs on both trees and only the difference is charged to the pull request (Figure 1). For LSP, ISP and DIP, which concern relations between units, a violation is charged when either end of the relation is in a changed file. A pull request that widens a base type is charged for subclasses it breaks, even in files it did not change. We call a violation *present* if it exists in the code the pull request touched, whoever wrote it, and *introduced* if the before/after comparison attributes it to the pull request.

3.6 Labelling

For each principle and language we sampled 100 entities the detector flagged and 100 it did not, from production code (test and generated code excluded), with a fixed seed. An entity touched by several pull requests was counted once. When a detector flagged few entities, all of them were labelled: the 195 Java DIP flags and the 80 Python ISP flags. The labels were produced by an LLM of the Claude family used as a judge [30]. Java entities were labelled between 5 and 8 September 2026 and Python entities on 14 September 2026. The judge read the source of each entity (for LSP, the overriding method next to its base method), did not see the detector's verdict, and returned a yes/no label with a written reason, following a rubric written before labelling began. In Python, borderline decisions were recorded as numbered precedents and applied to later entities. The rubric is part of the replication package. Labels could only be recorded through a script that rejects a label without a reason or without the source file shown to the judge, and every batch was checked by an automatic audit before labelling. All precision and recall values in this paper measure agreement between a detector and this judge.

4 Results

We report RQ2 first, because the RQ1 analyses use the precision values it produces to adjust flag counts.

4.1 RQ2: Measurability

Motivation. The SOLID principles are about intent, and the detectors see only structure. The rates reported for RQ1 are only meaningful if we know how often a flag is a real violation and how many violations the detectors miss.

Approach. Precision is the share of labelled flags that the judge confirmed. Since each sample is half flagged and half unflagged while the population is not, recall is weighted by the size of each stratum. With N_f and N_u the numbers of flagged and unflagged

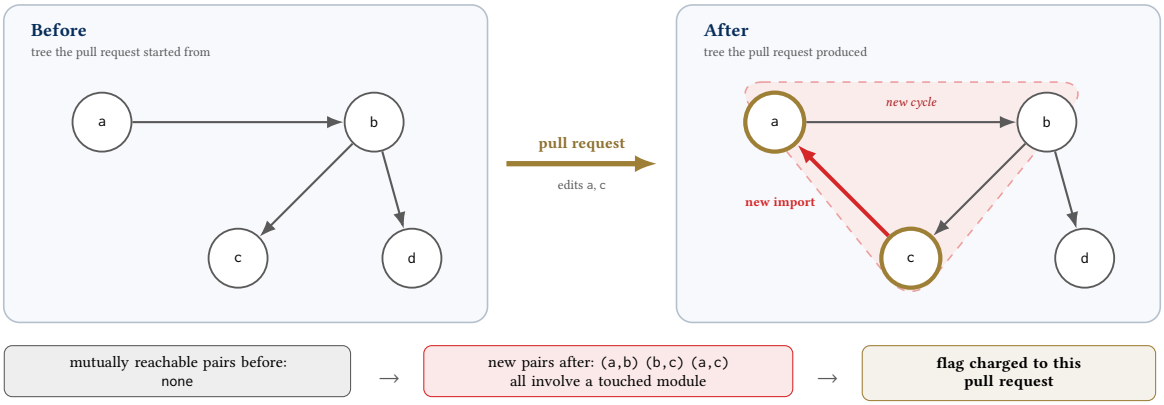


Figure 1: Before/after comparison for Dependency Inversion. The pull request edits modules `a` and `c` and adds the import `c→a`, which makes three module pairs mutually reachable. Only this difference is charged to the pull request.

entities in the population and n_f and n_u the numbers sampled,

$$R = \frac{TP \cdot N_f/n_f}{TP \cdot N_f/n_f + FN \cdot N_u/n_u}. \quad (1)$$

Confidence intervals come from a percentile bootstrap within each stratum. After the first Java round, we derived two refinements from the false positives: LSP restricted to refused bequest, and ISP restricted to contracts with at least 30 members. Both were re-scored on a holdout of 200 new entities drawn with a different seed, excluding every entity labelled before. Both refinements were fixed before any Python label existed, so the Python labels give a second test. Independently of the judge, we ran CK 0.7.0 [1], PMD 7.7.0 [23] and DesigniteJava 1.1.2 [25] with their default settings on the latest tree of 150 randomly chosen Java repositories, and measured agreement with Cohen's κ [5], without treating any tool as correct. To check the judge's consistency, 174 Java entities were labelled twice; the two labels agreed in 89.7% of cases.

Results. Table 4 and Figure 2 show how each detector agreed with the judge on 1,495 Java and 980 Python entities.

Dependency Inversion. All 195 Java flags were labelled, so the Java precision of 0.892 (174 of 195) is the value for the whole corpus, not an estimate. In Python, each of the 100 labelled flags closes a new import cycle through changed code. This shows that the cycles exist, not that every one of them is harmful.

Liskov Substitution. The three Java signals differ in precision. Refused bequest was correct on 31 of 31 flags in the first sample and 33 of 33 in the holdout, 64 of 64 overall (1.000, 95% interval 0.943–1.000). Weakened postcondition was correct on 6 of 36 flags and strengthened precondition on 3 of 33. The Python detector, with five signals, reached 0.510. In 46 of its 51 true positives the violation is a removed parameter, an added required parameter or a narrowed parameter type, all of which `javac` would reject. The refused-bequest rule, applied to Python without changes, was correct on six of seven flags.

Interface Segregation. Java precision depends on contract width: 0.116 for up to 17 members, 0.071 for 18 to 29, 0.786 for 30 to 49 and 0.900 for 50 to 99. A minimum of 30 members gave 0.862 (25 of 29) on the sample it was derived from, but 0.704 (19 of 27) on the holdout, which is outside the holdout's interval of 0.515–0.841, and 0.200 (1 of 5) in Python. A possible explanation lies in the form of the two rules. Refused bequest is a yes/no property of a method body, while the value 30 was chosen where the false positives in one sample stopped, so it may partly fit the noise in that sample. The Python figures rest on 7 and 5 flags and only point in the same direction.

Table 4: Detector agreement with the judge. Flagged and unflagged entities were labelled separately; precision is TP over flagged entities labelled, and recall is weighted by stratum size (Equation 1).

		Flagged		Unflagged			
	Principle	n	TP	n	FN	Prec.	Weighted recall
Java	SRP	100	35	100	1	0.350	0.828 (0.579–1.000)
	OCP	100	33	100	2	0.330	0.742 (0.512–1.000)
	LSP	200	84	200	3	0.420	0.386 (0.210–1.000)
	ISP	200	67	200	9	0.335	0.464 (0.331–0.680)
	DIP[a]	195	174	100	0	0.892	1.000[b]
Python	SRP	100	11	100	0	0.110	1.000[b]
	OCP	100	39	100	13	0.390	0.364 (0.257–0.523)
	LSP	100	51	100	2	0.510	0.545 (0.315–1.000)
	ISP[a]	80	34	100	0	0.425	1.000[b]
	DIP	100	100	100	0	1.000	1.000[b]

TP: flags the judge confirmed. FN: unflagged entities the judge labelled a violation. [a] All flags labelled (195 Java DIP pull requests, 80 Python ISP contracts). [b] No unflagged entity was labelled a violation, so no interval is given. Java LSP and ISP include the holdout round, which doubles their samples.

Open/Closed. Precision does not depend on the length of the chain. Across five bands of branch count it stays between 0.270 and 0.400. Most false positives switch over sets that are fixed by an external specification, such as JDBC type codes, protobuf scalar types, Unicode blocks or generated parser tables. The compiler treats these sets as open, so the enumeration rule does not suppress them.

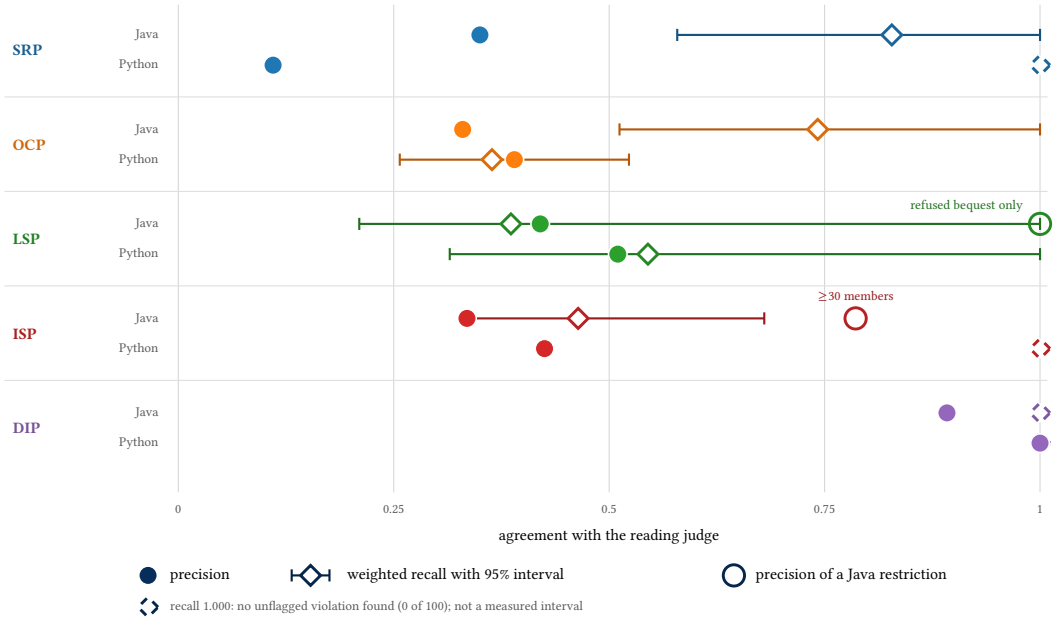


Figure 2: Detector precision and stratum-weighted recall against the judge, in Java and Python. Hollow rings show the precision of the two Java refinements.

Single Responsibility. Precision is 0.350 in Java and 0.110 in Python. Most of the 89 Python false positives are classes whose size is dictated by a framework, such as model heads in the `transformers` library, Home Assistant entities and Lightning callbacks.

Recall. In Java the SRP, OCP and DIP detectors miss few violations (weighted recall 0.742 to 1.000), so their flag rates are upper bounds on what the judge would confirm. The LSP and ISP detectors have lower recall, 0.386 and 0.464. The three Liskov violations among unflagged Java entities do not throw where the detector looks: one override throws for every non-null argument, one has an empty body, and one discards its argument. The weighting matters. For Java LSP the population contains 574 flagged and 25,594 unflagged override pairs, and without weighting the recall would be 0.966.

Precision by agent. Table 5 shows precision for each agent. In Java, Claude Code's flags were confirmed more often than Copilot's for four of the five principles, the exception being DIP (0.792 against 0.959). In Python, Claude Code's Open/Closed flags, which carry the agent difference reported in Section 4.2.1, had the highest precision of the three agents. Because the judge and Claude Code come from the same model family, we looked for signs of leniency. A lenient judge would confirm fewer of Claude Code's flags, and we did not see this.

Comparison with existing tools. Table 6 compares each Java detector with the closest rule or smell in the three tools. Over 82,553 production types, our SRP rubric agreed with PMD's `GodClass` rule at $\kappa = 0.574$ and with DesigniteJava's *Insufficient Modularization* and *Multifaceted Abstraction* smells at 0.531, while PMD and DesigniteJava agreed with each other at 0.384. Of the rubric's 6,326 flags, 4,304 (68.0%) were also flagged by at least one of the tools. Our size metrics correlated strongly with CK (Spearman $\rho = 0.977$ for weighted methods and 0.944 for lines of code), but cohesion did not (0.288 with CK's LCOM1, and CK's own two variants correlate at only 0.490). The Open/Closed detector found little support. Over 63,326 types in 138 repositories, it flagged 886 types (1.40%) and DesigniteJava's type-conditional smells flagged 299 (0.47%), with 76 types in common ($\kappa = 0.122$). For OCP, the judge is effectively the only check. The LSP and DIP differences come from how the units are defined. DesigniteJava reports a broken substitution on the supertype and a cycle on packages, while our detectors report the subclass and the compilation units. When our DIP flags are grouped by package, their recall against DesigniteJava is 0.924.

Table 5: Precision by agent: confirmed flags over labelled flags.

	Java			Python		
Principle	**Claude**	**Copilot**	**Codex**	**Claude**	**Copilot**	**Codex**
SRP	18/55	10/38	7/7	6/46	2/26	3/28
OCP	23/68	7/25	3/7	20/45	7/19	12/36
LSP	53/106	22/68	9/26	29/73	11/15	11/12
ISP	35/83	26/93	6/24	23/48	6/17	5/15
DIP	57/72	94/98	23/25	53/53	16/16	31/31

Table 6: Agreement of the Java detectors with existing analysers (Cohen's κ).

Principle	Compared with	κ
SRP	PMD `GodClass`; DesigniteJava modularization smells	0.574; 0.531 (the two tools with each other: 0.384)
OCP	DesigniteJava *Missing Hierarchy*, *Unexploited Encapsulation*	0.122
LSP	DesigniteJava *Rebellious Hierarchy*	0.000 (the tool reports the supertype, we report the subclass)
ISP	no counterpart in the three tools	not computable
DIP	DesigniteJava cyclic dependency	0.389 by compilation unit; 0.080 by package

Answer to RQ2. Partly, and the answer differs by principle. The DIP detector (new import cycles) and the refused-bequest rule for LSP have high precision in both languages. The SRP, OCP and ISP detectors have precision between 0.110 and 0.425, so their flags point to code that needs a closer look and should not be counted as violations. The SRP rubric agrees with published God Class detectors about as well as they agree with each other, while the OCP detector agrees with none of the tools. These values describe agreement with an LLM judge and should not be read as accuracy against ground truth.

4.2 RQ1: Adherence and differentiation

RQ1 has three parts: how the agents differ (Section 4.2.1), whether merged code differs from unmerged code (Section 4.2.2), and how many of the violations in the touched code a pull request introduced (Section 4.2.3).

4.2.1 *Differences between agents.*

Motivation. Teams choosing a coding agent may want to know whether the agents differ in the structure of the code they contribute, and whether such a difference comes from the agent or from the kind of change it makes.

Approach. We report flag rates per agent as the average over repositories (macro) and pooled over all pull requests, because one large repository can dominate a pooled rate. Proportions have Wilson intervals [29]. To separate the agent from change size, repository and time, we fitted for each principle and language the model

$$\begin{aligned}\operatorname{logit}\Pr(\text{flag}) &= \beta_0 + \beta_{\text{agent}} \\ &\quad + \beta_1 \log(1+\text{files}) + \beta_2\,\text{months},\end{aligned} \tag{2}$$

using generalised estimating equations [14] with an exchangeable working correlation within repositories and cluster-robust standard errors. Copilot, the largest Java group, is the reference agent. Each agent coefficient is compared with a Bonferroni threshold over the agent coefficients in its language: six in Java ($0.05/6 = 0.0083$) and ten in Python ($0.05/10 = 0.005$).

Results. Table 7 gives the flag rates. Claude Code has the highest pooled rate for every principle in both languages.

The agents make changes of very different sizes. On average a pull request touches 46.4 files for Claude Code, 18.5 for Copilot and 10.9 for Codex in Java, and 16.5, 7.7 and 9.6 files in Python. Flag rates increase with size for every agent, which is expected for detectors that compare two trees: a larger change gives them more opportunities to fire. When Java pull requests are grouped by size, the ordering of the agents only holds for the largest changes. Among pull requests that touch 11 or more files, 31.3% of Claude Code's are flagged for OCP, compared with 8.9% of Codex's. Among pull requests that touch three files or fewer, Copilot has the highest OCP and ISP rates and Codex the highest DIP rate.

Table 7: Flag rate by agent (%): average over repositories, with the pooled rate in brackets.

	Principle	Claude Code	Copilot	Codex
Java	PRs	1,109	2,366	770
	SRP[a]	8.4 (9.5)	3.5 (5.1)	7.4 (6.8)
	OCP	10.1 (10.6)	3.0 (6.4)	2.2 (2.1)
	LSP[b]	0.3 (0.7)	0.1 (0.3)	0.0 (0.0)
	ISP	6.0 (8.7)	4.0 (6.7)	5.0 (2.9)
	DIP	5.1 (6.5)	2.8 (4.1)	5.4 (3.2)
Python	PRs	1,662	1,470	1,814
	SRP[a]	11.0 (9.6)	10.3 (6.6)	8.1 (8.9)
	OCP	9.6 (9.7)	3.1 (3.5)	6.6 (5.7)
	LSP	1.2 (1.6)	0.3 (0.5)	0.8 (0.8)
	ISP	1.0 (1.3)	0.4 (0.4)	0.1 (0.3)
	DIP	5.6 (5.8)	1.5 (2.0)	2.3 (2.6)

[a] Over classes created by the agent; other rows over pull requests. [b] Refused bequest only, the rule used for Java (16 pull requests in total).

Table 8 shows the fitted model. Change size is the strongest term for every principle in both languages. Each doubling of the number of files touched multiplies the odds of a flag by 1.88 to 1.95 in Java and by 2.01 to 2.18 in Python ($p < 10^{-4}$).

Only one agent effect appears in both languages. Claude Code's odds of an OCP flag are 2.22 times Copilot's in Java (1.21–4.07, $p = 0.010$) and 2.09 times in Python (1.44–3.04, $p = 0.0001$). The Java estimate on its own does not pass the Bonferroni threshold of 0.0083, but the Python estimate passes 0.005, and the two estimates are close. We therefore consider this difference supported. In Python, Claude Code also has higher odds of introducing an import cycle (1.88, 1.17–3.03, $p = 0.0096$). This estimate does not pass the Bonferroni threshold, and the Java estimate (1.29) has an interval that includes 1, so we do not treat it as confirmed. All other agent coefficients have intervals that include 1.

Time had no detectable effect on the before/after principles in either language. The Python SRP flag at pull-request level increases by 5% per month ($p < 10^{-4}$). Since this flag has a precision of 0.110, the trend tells us that large classes are touched or created more often over time, not that responsibility violations are increasing.

4.2.2 *Merged and unmerged code.*

Motivation. Most studies of agent code use only merged pull requests. If maintainers tended to reject structurally worse code, results from merged code would partly reflect the reviewers' choices.

Approach. We compared only pull requests recovered with methods 2 to 4, so that merged and unmerged pull requests obtain their before tree in the same way, and we split unmerged pull requests by how they were closed. In Python we also added an indicator for unmerged pull requests to the model in Equation (2).

Results. Table 9 gives the flag rates for each group.

In Java, merged pull requests and the 1,006 pull requests closed without review have rates within two percentage points of each other for every principle. The two reviewed groups have higher ISP

Table 8: Adjusted odds ratios with 95% confidence intervals (reference agent: GitHub Copilot).

	Term	SRP	OCP	LSP	ISP	DIP
Java	Claude Code	n/a[a]	2.22 (1.21–4.07)	n/a[b]	1.11 (0.63–1.95)	1.29 (0.50–3.36)
	OpenAI Codex	n/a[a]	0.57 (0.24–1.37)	n/a[b]	0.72 (0.40–1.32)	1.68 (0.79–3.56)
	Size, per doubling	n/a[a]	1.89 (1.75–2.04)	n/a[b]	1.88 (1.73–2.04)	1.95 (1.77–2.16)
	Time, per month	n/a[a]	1.00 (0.91–1.10)	n/a[b]	1.02 (0.95–1.10)	1.03 (0.93–1.15)
	Flagged / total	n/a[a]	286 / 4,245	16 / 4,245	276 / 4,245	195 / 4,245
Python	Claude Code	1.09 (0.93–1.29)	2.09 (1.44–3.04)[†]	1.81 (0.74–4.42)	1.71 (0.67–4.36)	1.88 (1.17–3.03)
	OpenAI Codex	1.06 (0.90–1.25)	1.46 (0.98–2.16)	1.38 (0.54–3.52)	0.53 (0.16–1.80)	1.12 (0.67–1.86)
	Size, per doubling	2.06 (1.94–2.18)	2.01 (1.88–2.15)	2.18 (1.89–2.51)	2.17 (1.82–2.58)	2.04 (1.89–2.21)
	Time, per month	1.05	0.99	0.96	0.94	1.06
	Flagged / total	2,532 / 4,946	315 / 4,946	47 / 4,946	32 / 4,946	173 / 4,946

[a] Java SRP is scored per class, so there is no pull-request flag to model; created and edited classes are compared in Section 4.2.3. [b] Not fitted, because only 16 pull requests are flagged. [†] The only agent coefficient that passes the Bonferroni threshold of its language (Java $0.05/6 = 0.0083$, Python $0.05/10 = 0.005$); Java OCP for Claude Code has $p = 0.010$ and Python DIP for Claude Code $p = 0.0096$. Java model: 790 repositories; Python model: 2,921.

Table 9: Flag rate (%) by outcome, for pull requests recovered with methods 2 to 4.

	Outcome	n	OCP	LSP	ISP	DIP
Java	merged	1,521	8.3	0.5	7.5	4.6
	changes requested	36	5.6	0.0	16.7	13.9
	reviewed, closed	135	12.6	0.7	12.6	12.6
	closed unreviewed	1,006	7.5	0.5	5.5	5.6
	open	490	8.8	0.2	9.2	5.9
Python	merged	1,734	4.9	0.5	0.4	3.1
	changes requested	33	9.1	0.0	0.0	9.1
	reviewed, closed	163	7.4	1.2	1.8	8.6
	closed unreviewed	1,300	9.1	1.7	0.9	4.5
	open	553	10.8	2.2	1.1	5.2

SRP is scored per class and is not split by outcome. Python flagged counts for merged pull requests: OCP 85, LSP 8, ISP 7, DIP 54.

Table 10: Violations present in the touched code and pull requests that introduced one.

	Java		Python	
Principle	Present (adj.)	Introd.	Present (adj.)	Introd.
SRP[a]	7,501 (≈2,600)	n/a	8,208 (≈900)	449 (1.0%)
OCP	2,343 (≈770)	286 (2.2%)	2,580 (≈1,000)	315 (2.5%)
LSP[b]	183 (183)	16 (0.4%)	176	5
ISP	816 (≈270)	276 (2.2%)	80 (34)	32 (0.3%)
DIP[c]	195 (174)	195 (4.1%)	173 (≈173)	173 (3.5%)

Present: distinct flagged production entities in the touched code, with the precision-adjusted estimate in brackets. Introduced: flagged pull requests, with the precision-adjusted share of all analysable pull requests in brackets. [a] Classes; Java has no introduced count because its SRP flag is per class. [b] Refused-bequest relationships in both languages; with all five Python signals, 2,069 override pairs (≈1,050 adjusted) and 47 pull requests (0.5%). [c] The detector records only new cycles, so both counts are pull requests.

and DIP rates. For pull requests that were reviewed and then closed, the Wilson interval for DIP (8.0–19.2%) does not overlap the interval for merged code (3.7–5.8%). These groups are small, however, and 6 of the 17 flagged pull requests in the reviewed-then-closed group come from one repository.

In Python, merged pull requests have lower rates than the unreviewed, reviewed-then-closed and open groups for all four principles. For OCP, for example, the rate is 4.9% for merged pull requests and 9.1% for the 1,300 closed without review. Unmerged pull requests are larger (median four files compared with three). After adjusting for size and agent, with repository clustering, unmerged pull requests still have higher odds of an OCP flag (1.57, 1.15–2.15, $p = 0.004$) and of an LSP flag (2.43, 1.08–5.46, $p = 0.03$). For ISP (1.71, 0.75–3.91) and DIP (1.36, 0.92–2.03) the difference is not significant.

4.2.3 *Present and introduced violations.*

Motivation. A violation in the code an agent touched was not necessarily written by the agent. Comparing the violations a pull request introduced with those already present shows whether agents create design problems or mostly work in code that already has them.

Approach. The present count is the number of distinct flagged production entities in the touched code. The introduced count is the number of pull requests whose before/after comparison adds a violation. Multiplying both counts by the detector's precision gives precision-adjusted estimates. These assume that the precision measured on the sample holds for the whole corpus, and they carry the uncertainty of that measurement. For SRP we compare classes created by the agents with existing classes they edited.

Results. Table 10 and Figure 3 compare the two counts.

According to the precision-adjusted estimates, between 0.4% and 4.1% of Java pull requests and between 0.3% and 3.5% of Python pull requests introduced a violation of a given principle. The touched code already contained hundreds to thousands of flagged entities. The difference is largest for the principles that involve two units. The touched Java code contains 183 refused-bequest relationships, and 16 pull requests introduced one. In Python the numbers are 176 and five. A refused bequest needs a base method and an override that contradicts it, an ISP violation needs a contract and its implementers, and a cycle needs several modules. A single pull request rarely writes both sides of such a relation. An SRP or OCP violation, by contrast, sits inside one class or one conditional, which a single pull request can write completely.

The counts for refused bequest are small (16 Java and five Python pull requests), so the introduction rate for LSP is imprecise and we report it only as an order of magnitude. The LSP detector also has the lowest recall of the five and undercounts in the 718 Java pull requests with less than 50% of supertypes resolved. Both problems mean that more violations are present than we counted, so the real gap is probably larger.

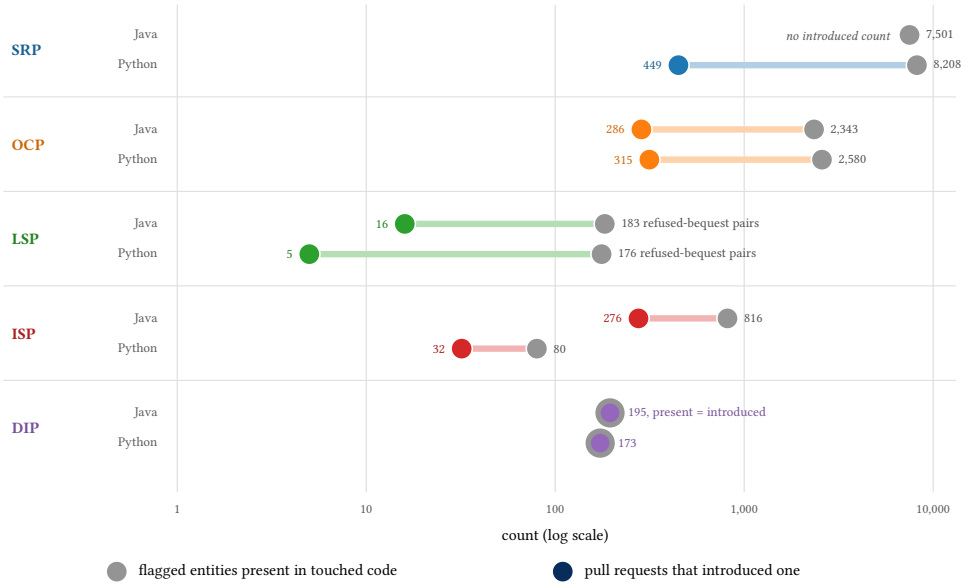


Figure 3: Flagged violations present in the production code the agents touched (grey) and pull requests that introduced one (colour), on a log scale. LSP uses refused bequest in both languages.

Classes created by the agents are flagged less often than existing classes they edited. Pooled, the rates are 7.7% and 13.5% in Java (13,712 created and 48,433 edited classes) and 8.9% and 12.1% in Python. Averaged over repositories they are 5.6% and 20.9% in Java and 9.7% and 23.6% in Python. New classes also tend to be smaller and younger than existing ones, so this comparison describes the corpus and should not be read as a comparison between agents and human developers.

Answer to RQ1. The agents do introduce SOLID violations, but rarely. The precision-adjusted share of pull requests that introduce one is 0.4–4.1% in Java and 0.3–3.5% in Python, depending on the principle, while the code they touch already contains many more. Violations between two units mostly build up over many changes. The agents differ less than raw rates suggest: change size accounts for most of the differences, and only Claude Code's higher OCP rate holds in both languages. These are adjusted associations, since the agents worked on different tasks. Merged code does not differ from unreviewed code in Java but has fewer flags in Python; with only 171 and 196 reviewed rejections, the data cannot show what reviewers base their decisions on.

5 Discussion

5.1 Implications for practitioners

The differences between the three agents are small and mostly related to how much code each agent changes in one pull request. A SOLID profile can complement functional benchmarks when choosing an agent, but it does not replace them.

For teams that run automated checks on pull requests, refused bequest is a candidate check. It is simple to compute and was confirmed on 64 of 64 Java flags and six of seven Python flags. It is also rare: only 16 of 4,245 Java pull requests introduced one, and we did not measure the recall of this rule on its own, so it would catch few problems. In Python, a check that an override still accepts the calls its base method accepts may be more useful, since such faults made up 46 of the 51 confirmed Python Liskov violations. We would not reuse an interface width threshold from another corpus. Ours lost 16 points on the holdout and was correct on one of five flags in Python. Size-based SRP flags and branch-count OCP flags are useful as prompts for a reviewer, but should not block a merge, since most of them were not confirmed by the judge. Problems between units, which build up over many changes, are hard to see when reviewing one pull request at a time. Checks that look at the history of a component would be better suited to them.

5.2 Implications for researchers

Validating detectors. Detector precision in this study ranged from 0.110 to 1.000. Taking flags at face value would overstate Python SRP violations about ninefold (11.6% of touched classes are flagged, 1.3% precision-adjusted). For Java LSP, unweighted recall would be 2.5 times the weighted value (0.966 compared with 0.386). Studies that rely on static detectors should report their precision and weight recall by the sampled population.

Effect of the language. Three kinds of Liskov fault that made up most of the confirmed Python violations cannot occur in Java code that compiles. A detector designed for one language may cover only part of the principle in another, and comparing raw rates across languages mixes differences between agents with differences between type systems.

Refining detectors. Both refinements were derived from the same Java labels. The rule based on a code pattern held on the Java holdout and the numeric threshold did not. This is one pair of rules, so it is weak evidence, but it suggests testing any threshold tuned on labelled false positives on a fresh sample before using it.

Combining detectors and language models. Pehlivan et al. [22] found that language models detect SRP violations most easily and DIP violations least easily. Our structural detectors show the opposite order: DIP has the highest precision (0.892 and 1.000) and SRP the lowest (0.350 and 0.110). A model reading a short example can judge whether a class does several things, but it cannot see an import cycle across files it was not shown. A static analyser computes the whole import graph but cannot judge intent. Using both kinds of detector together might cover more of the five principles than either one alone; we did not test this. The two studies also differ in data and setting, so the comparison is indirect.

5.3 Comparison with earlier studies

The strong effect of change size is consistent with the Volume-Quality Inverse Law reported by Zhu et al. [32], here observed on real pull requests, in two languages and for all modelled principles. It also agrees with Cynthia et al. [6], who found that differences between agents in SonarQube issues mostly disappear after normalising for churn, although we found one difference that remains. The similarity between merged and unreviewed Java code is in line with METR's observation that passing tests and acceptance by maintainers measure different things [28]. NITR [31] found that agents rarely succeed in repairing dependency structure, and we found that they rarely damage it. The two results are consistent with agents changing dependency structure little in either direction, although neither study tests this directly. SlopCodeBench [21] measured erosion over long sequences of edits by the same agent. Our unit is a single pull request, so our time coefficient does not test that result.

6 Threats to Validity

Construct validity. We measure conformity to five principles, not design quality, and we do not observe outcomes such as defects or maintenance cost. The link between smells and such outcomes is not uniform [20]. Each detector is a proxy, and each agreement value is agreement with an LLM judge, which can make the same mistakes as a detector. The comparison with three existing tools does not depend on the judge, but it covers only Java and has no counterpart for ISP. The judge's consistency was measured only in Java (89.7% agreement on 174 entities labelled twice). The judge and Claude Code come from the same model family. Precision by agent shows no sign of leniency, but it would not reveal a bias shared by the judge and a detector. The Python ISP flag is defined differently from the Java one, so the raw ISP rates (6.5% of Java and 0.6% of Python pull requests) cannot be compared. For Python DIP, the labels confirm that the flagged cycles exist, not that they are harmful. The detector thresholds are percentile-based screening choices, and their only support is the precision and recall measured for RQ2.

Internal validity. The agents chose their own tasks in repositories we do not control, so the model coefficients are adjusted associations and not causal effects. Differences in the tasks given to each agent remain in its coefficient. Adjusting for change size also removes any part of an agent's effect that works through the size of its changes. Change size is measured as the number of files touched. We did not test other measures, such as lines or classes changed, so the size effect and the adjusted agent differences may depend on this choice. Time is modelled as a linear trend, which would not detect a sudden change such as a new model release. Symbol resolution is incomplete (median 87.0% in Java, and only bases inside the repository in Python), which removes candidates and makes the LSP and ISP rates in RQ1 lower bounds. Because a relation-level violation is charged when either end is touched, the present counts in Section 4.2.3 are an upper bound on what the agents could be responsible for. The outcome comparison in Section 4.2.2 uses only methods 2 to 4, since method 1 exists only for merged pull requests, and it is also affected by the uneven loss of deleted branches described below.

External validity. Both corpora come from one dataset and from repositories with at least 25 stars and a permissive licence. Repositories above the parsing limit were excluded (150 Java pull requests). Java and Python both use nominal class hierarchies, so languages with structural typing are not covered. The Python corpus lost 496 pull requests (7.0%) because their branches were deleted. The loss was uneven: 11.0% of rejected and 5.1% of merged pull requests, and 14.3% of Copilot's pull requests compared with 3.6% and 3.1% for the other two agents. We did not replace them, because replacements could only come from pull requests whose branches still exist. The agents were updated during the period covered by the data, and the pull requests of each agent come from different months.

Conclusion validity. Several agent coefficients are tested, so we apply a Bonferroni correction per language. Counts for the relation-level principles are small (16 Java pull requests introduce a refused bequest, and 32 Python pull requests are flagged for ISP), which gives wide intervals. Weighted recall depends on the few violations found in large unflagged strata. Only 171 Java and 196 Python unmerged pull requests were reviewed, which limits what the outcome comparison can say about review.

7 Conclusion

Within the limits of our detectors, the three coding agents rarely introduced SOLID violations in the 9,191 pull requests we analysed. The precision-adjusted share of pull requests that introduced a violation was between 0.3% and 4.1% per principle and language, and the code they touched already contained many more flagged problems. Differences between the agents were mostly accounted for by the number of files they changed, and only Claude Code's higher OCP rate appeared in both languages.

The main qualification is the strength of the detectors. The evidence is best for import cycles and refused bequest, which had high precision against the judge. For SRP, OCP and ISP the flags are screens, and the estimates depend on precision measured against an LLM judge. For automated review, refused bequest is precise but rare, while size-based and branch-count flags are better treated as hints for reviewers.

Future work could add a change-history signal for OCP, recognise sets fixed by external specifications, make the Python SRP screen aware of frameworks, add a second judge from a different model family, and follow repositories over time to study violations that build up across many pull requests.

Data Availability

The corpus is derived from the public AIDev dataset [13]. The pipeline, configuration, result tables, the 2,475 labels with their reasons and the scripts that compute every reported number are kept under version control; the replication package is linked in Section 1.

Ethics

The study involves no human participants and no personal data. All repositories were public and had a permissive licence when they were collected, and they are not redistributed. A language model was used only to read public source code and label a validation sample. It does not compute any reported number.

References

[1] Maurício Aniche. 2015. Java Code Metrics Calculator (CK). https://github.com/mauricioaniche/ck/.

[2] Oussama Ben Sghaier, Hao Li, Bram Adams, and Ahmed E. Hassan. 2026. Don't Blame the Large Language Model: How Agent Harness Evolution Shapes Coding Agent Quality. *arXiv preprint arXiv:2607.03691* (2026). https://arxiv.org/abs/2607.03691

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). https://arxiv.org/abs/2107.03374

[4] Shyam R. Chidamber and Chris F. Kemerer. 1994. A Metrics Suite for Object Oriented Design. *IEEE Transactions on Software Engineering* 20, 6 (1994), 476–493.

[5] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104

[6] Shamse Tasnim Cynthia, Al Muttakin, and Banani Roy. 2026. Beyond Bug Fixes: An Empirical Investigation of Post-Merge Code Quality Issues in Agent-Generated Pull Requests. *arXiv preprint arXiv:2601.20109* (2026). https://arxiv.org/abs/2601.20109

[7] Martin Fowler, Kent Beck, John Brant, William Opdyke, and Don Roberts. 1999. *Refactoring: Improving the Design of Existing Code.* Addison-Wesley.

[8] Martin Hitz and Behzad Montazeri. 1995. Measuring Coupling and Cohesion in Object-Oriented Systems. In *Proceedings of the International Symposium on Applied Corporate Computing.* Monterrey, Mexico.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR).* https://arxiv.org/abs/2310.06770

[10] Syed Mohammad Kashif, Ruiyin Li, Peng Liang, Amjed Tahir, Qiong Feng, Zengyang Li, and Mojtaba Shahin. 2026. Beyond Functional Correctness: Design Issues in AI IDE-Generated Large-Scale Projects. *arXiv preprint arXiv:2604.06373* (2026). https://arxiv.org/abs/2604.06373

[11] Foutse Khomh, Massimiliano Di Penta, Yann-Gaël Guéhéneuc, and Giuliano Antoniol. 2012. An Exploratory Study of the Impact of Antipatterns on Class Change- and Fault-Proneness. *Empirical Software Engineering* 17 (2012), 243–275.

[12] Phongsakon Mark Konrad, Tim Lukas Adam, Riccardo Terrenzi, and Serkan Ayvaz. 2026. Architecture Without Architects: How AI Coding Agents Shape Software Architecture. *arXiv preprint arXiv:2604.04990* (2026). https://arxiv.org/abs/2604.04990

[13] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025). https://arxiv.org/abs/2507.15003

[14] Kung-Yee Liang and Scott L. Zeger. 1986. Longitudinal Data Analysis Using Generalized Linear Models. *Biometrika* 73, 1 (1986), 13–22.

[15] Andrian Marcus, Denys Poshyvanyk, and Rudolf Ferenc. 2008. Using the Conceptual Cohesion of Classes for Fault Prediction in Object-Oriented Systems. *IEEE Transactions on Software Engineering* 34, 2 (2008), 287–300.

[16] Radu Marinescu. 2004. Detection Strategies: Metrics-Based Rules for Detecting Design Flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM).* 350–359.

[17] Robert C. Martin. 2000. Design Principles and Design Patterns. Object Mentor, white paper.

[18] Robert C. Martin. 2003. *Agile Software Development: Principles, Patterns, and Practices.* Prentice Hall.

[19] Iren Mazloomzadeh, Mohammad Mehdi Morovati, and Foutse Khomh. 2026. How Do AI Coding Agents Contribute to Software Development? An Empirical Study of Agentic Pull Requests. *arXiv preprint arXiv:2607.21832* (2026). https://arxiv.org/abs/2607.21832

[20] Steffen M. Olbrich, Daniela S. Cruzes, and Dag I. K. Sjøberg. 2010. Are All Code Smells Harmful? A Study of God Classes and Brain Classes in the Evolution of Three Open Source Systems. In *Proceedings of the IEEE International Conference on Software Maintenance (ICSM).*

[21] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. 2026. SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks. *arXiv preprint arXiv:2603.24755* (2026). https://arxiv.org/abs/2603.24755

[22] Fatih Pehlivan, Arçin Ülkü Ergüzen, Sahand Moslemi Yengejeh, Mayasah Lami, and Anil Koyuncu. 2025. Are We SOLID Yet? An Empirical Study on Prompting LLMs to Detect Design Principle Violations. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE).* https://arxiv.org/abs/2509.03093 arXiv:2509.03093.

[23] PMD contributors. 2024. PMD: An Extensible Cross-Language Static Code Analyzer. Version 7.7.0, https://pmd.github.io.

[24] Shota Sawada, Tatsuya Shirai, Yutaro Kashiwa, Ken'ichi Yamaguchi, Hiroshi Iwata, and Hajimu Iida. 2026. To What Extent Does Agent-Generated Code Require Maintenance? An Empirical Study. In *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE).* https://arxiv.org/abs/2605.06464 arXiv:2605.06464.

[25] Tushar Sharma, Pratibha Mishra, and Rohit Tiwari. 2016. Designite: A Software Design Quality Assessment Tool. In *Proceedings of the 1st International Workshop on Bringing Architectural Design Thinking into Developers' Daily Activities (BRIDGE).*

[26] Girish Suryanarayana, Ganesh Samarthyam, and Tushar Sharma. 2014. *Refactoring for Software Design Smells: Managing Technical Debt.* Morgan Kaufmann.

[27] Robert Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160.

[28] Parker Whitfill, Cheryl Wu, Joel Becker, and Nate Rush. 2026. Many SWE-bench-Passing PRs Would Not Be Merged into Main. METR research note, 10 March 2026. https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/

[29] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

[30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS).* https://arxiv.org/abs/2306.05685

[31] Haichao Zhu, Qian Zhang, Jiyuan Wang, Zhaorui Yang, and Yuxin Qiu. 2026. Needle in the Repo: A Benchmark for Maintainability in AI-Generated Repository Edits. *arXiv preprint arXiv:2603.27745* (2026). https://arxiv.org/abs/2603.27745

[32] Yuecai Zhu, Nikolaos Tsantalis, and Peter C. Rigby. 2026. AI-Generated Smells: An Analysis of Code and Architecture in LLM and Agent-Driven Development. *arXiv preprint arXiv:2605.02741* (2026). https://arxiv.org/abs/2605.02741

[33] Terry Yue Zhuo et al. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations (ICLR).* https://arxiv.org/abs/2406.15877